

K-CUBE API Specification

Prepared: 2026-05-30

Purpose

This document defines how the React/Next.js frontend communicates with the Node.js/Express backend. It includes the current API foundation plus the required Google Workspace integration plan for Google Single Sign-On and Google Calendar event sync.

Base path:

```
/api/v1
```

Auth header:

```
Authorization: Bearer <accessToken>
```

Common success shape:

```
{
  "success": true,
  "data": {}
}
```

Common error shape:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Human-readable message"
  }
}
```

Frontend Integration Architecture

The attached `App.js` concept should be integrated as route intent, not as a literal file, because the current frontend uses Next.js App Router.

Frontend concern	Current project location
API client	<code>frontend/src/lib/api.ts</code>
Query/cache provider	<code>frontend/src/lib/queryClient.ts</code> , <code>frontend/src/components/Providers.tsx</code>
Public routes	<code>frontend/src/app/*/page.tsx</code>
Protected member dashboard	<code>frontend/src/app/dashboard/page.tsx</code>
Admin area	<code>frontend/src/app/admin/page.tsx</code>

Frontend rules:

- Public pages may call public content APIs without a token.
- Member actions send `Authorization: Bearer <accessToken>`.
- On `401`, frontend calls `/auth/refresh`, then retries once.
- Admin pages must check `user.role === "admin"` after `/users/profile`.
- Google Sign-In button starts Google Identity Services flow and sends a backend-verifiable credential/code to K-CUBE.

Health

Method	Path	Auth	Description
GET	<code>/health</code>	Public	Backend process health
GET	<code>/api/v1/health</code>	Public	API health

Authentication API

Endpoint Summary

Method	Path	Auth	Status	Description
POST	<code>/auth/register</code>	Public	Existing	Email/password registration
POST	<code>/auth/login</code>	Public	Existing	Email/password login
POST	<code>/auth/otp/send</code>	Public	Existing dev foundation	Send phone OTP
POST	<code>/auth/otp/verify</code>	Public	Existing dev foundation	Verify phone OTP
POST	<code>/auth/google</code>	Public	Must upgrade	Google SSO callback/handoff
GET	<code>/auth/verify</code>	Bearer	Existing	Validate access token
POST	<code>/auth/refresh</code>	Public	Existing	Refresh access token
POST	<code>/auth/logout</code>	Bearer	Proposed	Invalidate refresh session

Register

```
POST /api/v1/auth/register
```

Request:

```
{
  "full_name": "K-CUBE Member",
  "username": "kcube_member",
  "email": "member@example.com",
  "phone": "+910000000000",
  "password": "secret",
  "category_access": "category_c",
  "referral_code": "KC-ABCDEFGH"
}
```

Response:

```
{
  "success": true,
  "data": {
    "user": {
      "id": 1,
      "email": "member@example.com",
      "username": "kcube_member",
      "full_name": "K-CUBE Member",
      "role": "member",
      "category_access": "category_c",
      "points": 250,
      "referral_code": "KC-ABCDEFGH"
    },
    "token": "<access-jwt>",
    "refreshToken": "<refresh-jwt>"
  }
}
```

Google SSO

POST /api/v1/auth/google

Current state:

- The backend has a placeholder that accepts `google_id`, `email`, and `full_name`.
- This must be replaced before production because the browser must not be trusted to self-report Google identity.

Recommended production flow:

1. Frontend loads Google Identity Services. 2. User clicks "Continue with Google". 3. Frontend obtains either an ID token or an authorization code. 4. Frontend sends that credential/code to K-CUBE backend. 5. Backend verifies the ID token or exchanges the code server-side. 6. Backend reads verified claims: `sub`, `email`, `email_verified`, `name`, `picture`, hosted domain if needed. 7. Backend links/creates `users` and `auth_identities`. 8. Backend issues K-CUBE access and refresh JWTs.

Request option A, ID token:

```
{
  "credential": "<google-id-token>",
  "referral_code": "KC-ABCDEFGH"
}
```

Request option B, authorization code:

```
{
  "code": "<google-authorization-code>",
  "redirect_uri": "https://k-cube.store/auth/google/callback",
  "referral_code": "KC-ABCDEFGH"
}
```

Response:

```
{
  "success": true,
  "data": {
    "user": {
```

```
    "id": 12,
    "email": "member@gmail.com",
    "full_name": "K-CUBE Member",
    "profile_image": "https://...",
    "role": "member",
    "category_access": "category_c",
    "points": 250
  },
  "token": "<access-jwt>",
  "refreshToken": "<refresh-jwt>",
  "isNewUser": true
}
```

Validation and security:

- Reject tokens where `email_verified` is false.
- Verify `aud` matches `GOOGLE_CLIENT_ID`.
- Use Google `sub` as the stable identity key, not email.
- Link to an existing user by verified email only when the account is not already linked to another Google `sub`.
- Store provider identity in `auth_identities`.
- Optional Workspace restriction: if K-CUBE wants only a specific Google Workspace domain, enforce `hd`.

Environment variables:

```
GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=
GOOGLE_AUTH_REDIRECT_URI=
GOOGLE_ALLOWED_WORKSPACE_DOMAIN=
```

User API

Method	Path	Auth	Description
GET	<code>/users</code>	Admin/manager	List/filter users
GET	<code>/users/profile</code>	Authenticated	Current user's profile, points, role, access
PATCH	<code>/users/:id</code>	Admin	Update user fields

Profile response:

```
{
  "success": true,
  "data": {
    "id": 1,
    "full_name": "K-CUBE Member",
    "email": "member@example.com",
    "role": "member",
    "category_access": "category_c",
    "points": 250,
    "xp": 0,
    "level": 1,
    "referral_code": "KC-ABCDEFGH"
  }
}
```

Events and Google Calendar API

Event Endpoint Summary

Method	Path	Auth	Description
GET	/events	Public	Published event list
GET	/events/:slug	Public	Event detail
POST	/events/:id/rsvp	Authenticated	Register current user for event
DELETE	/events/:id/rsvp	Authenticated	Cancel RSVP
POST	/events/:id/check-in	Admin	Mark attendance and award points
POST	/admin/events	Admin	Create event
PATCH	/admin/events/:id	Admin	Update event
POST	/admin/events/:id/sync/google-calendar	Admin	Push event to Google Calendar
GET	/admin/google-calendar/connections	Admin	List calendar connections
POST	/admin/google-calendar/connections	Admin	Create/update calendar connection
POST	/admin/google-calendar/sync	Admin	Run queued sync jobs

Create Event

```
POST /api/v1/admin/events
```

Request:

```
{
  "title": "Korean Culture Workshop",
  "slug": "korean-culture-workshop",
  "description": "Introductory K-Culture event for K-CUBE members.",
  "category": "k_culture",
  "starts_at": "2026-06-15T11:00:00+05:30",
  "ends_at": "2026-06-15T13:00:00+05:30",
  "timezone": "Asia/Kolkata",
  "location_name": "Delhi",
  "location_address": "Venue address",
  "capacity": 100,
  "points_reward": 50,
  "status": "published",
  "sync_to_google_calendar": true
}
```

Response:

```
{
  "success": true,
  "data": {
    "id": 10,
```

```
"slug": "korean-culture-workshop",
"sync_status": "pending"
}
```

RSVP

```
POST /api/v1/events/10/rsvp
```

Response:

```
{
  "success": true,
  "data": {
    "event_id": 10,
    "status": "registered"
  }
}
```

Sync Event to Google Calendar

```
POST /api/v1/admin/events/10/sync/google-calendar
```

Request:

```
{
  "calendar_id": "primary",
  "send_updates": "all"
}
```

Response:

```
{
  "success": true,
  "data": {
    "event_id": 10,
    "google_calendar_event_id": "abc123",
    "google_calendar_html_link": "https://calendar.google.com/...",
    "sync_status": "synced"
  }
}
```

Google Calendar sync mapping:

K-CUBE field	Google Calendar event field
title	summary
description	description
starts_at , timezone	start.dateTime , start.timeZone
ends_at , timezone	end.dateTime , end.timeZone
location_name/address	location
online_meeting_url	description or conference data, if enabled
RSVP users	Optional attendees[] , only if policy allows sending invites

Recommended Calendar integration mode:

- Phase 1: Admin OAuth to a single K-CUBE shared calendar.
- Phase 2: Google Workspace service account with domain-wide delegation if K-CUBE needs centralized access to organization calendars without each user's consent.

Calendar scopes:

```
https://www.googleapis.com/auth/calendar.events
```

Use broader `https://www.googleapis.com/auth/calendar` only if calendar creation/list management is required.

Calendar sync behavior:

- Insert Google event when a K-CUBE event is published and `google_calendar_event_id` is empty.
- Update Google event when K-CUBE title/time/location/status changes.
- Cancel or delete Google event when K-CUBE event is cancelled, according to admin policy.
- Store failed sync attempts in `google_calendar_sync_jobs`.
- Use incremental sync tokens for pulling calendar-side changes if two-way sync is enabled.

Environment variables:

```
GOOGLE_CALENDAR_ID=
GOOGLE_CALENDAR_SERVICE_ACCOUNT_EMAIL=
GOOGLE_CALENDAR_PRIVATE_KEY=
GOOGLE_WORKSPACE_DELEGATED_ADMIN_EMAIL=
GOOGLE_CALENDAR_SYNC_MODE=admin_oauth
```

K-Food API

Method	Path	Auth	Description
POST	/engagement/kfood/click	Authenticated	Track outbound K-Food click
POST	/engagement/kfood/purchase-claim	Authenticated	Submit manual order claim
GET	/admin/kfood/claims	Admin	List K-Food claims
PATCH	/admin/kfood/claims/:id/review	Admin	Approve/reject claim and award points

Purchase claim request:

```
{
  "order_id": "KF-10001",
  "order_total": 1500,
  "coupon_code": "KCUBE10"
}
```

Learning API

Method	Path	Auth	Description
POST	/engagement/lessons/complete	Authenticated	Mark lesson complete and award points once

Request:

```
{
  "lesson_id": 1,
  "accuracy": 95
}
```

Content Upload API

Method	Path	Auth	Description
POST	/engagement/uploads	Authenticated	Submit user upload
GET	/engagement/uploads/me	Authenticated	List current user's uploads
GET	/admin/uploads	Admin	Review queue
PATCH	/admin/uploads/:id/review	Admin	Approve/reject and award points

Points API

Method	Path	Auth	Description
GET	/users/profile	Authenticated	Current balance
GET	/admin/points	Admin	Ledger listing
POST	/admin/points/adjust	Admin proposed	Manual adjustment

Every award must write `point_transactions` and update `users.points` in one database transaction.

Search API

Method	Path	Auth	Description
GET	/search?q=<term>&category=<category>	Public	Search indexed content

Admin API

Method	Path	Description
GET	/admin/dashboard	Dashboard metrics
PATCH	/admin/user/:id	Manage user fields
POST	/admin/chapter/:id/approve	Approve chapter
POST	/admin/announcement	Publish announcement
GET	/admin/analytics	Analytics summary
GET	/admin/uploads	Upload review queue
PATCH	/admin/uploads/:id/review	Upload review
GET	/admin/points	Points ledger
GET	/admin/kfood/claims	K-Food claim queue
PATCH	/admin/kfood/claims/:id/review	K-Food claim review

Security Requirements

- Use strong `JWT_SECRET` and `JWT_REFRESH_SECRET`.
- Store refresh tokens in httpOnly cookies or server-side hashed session records before production.
- Replace dev OTP response that returns `otpCode`.
- Restrict CORS to approved frontend domains.
- Lock admin update fields to an allowlist.
- Rate-limit login, OTP, Google SSO, RSVP, and K-Food claim endpoints.
- Log admin changes to `admin_audit_logs`.
- Never store Google access tokens in plaintext.

Implementation Phases

Phase 1, foundation:

- Add production Google SSO verification.
- Persist Google identity in `auth_identities`.
- Add `platform_events`, `platform_event_rsvps`, and Google calendar sync tables.
- Add event CRUD and RSVP APIs.

Phase 2, calendar sync:

- Configure Google Cloud OAuth client.
- Enable Google Calendar API.
- Build admin calendar connection screen.
- Push published K-CUBE events to shared Google Calendar.
- Log sync jobs and retries.

Phase 3, Workspace-grade operations:

- Add domain-wide delegation only if K-CUBE needs Workspace-wide calendar access.

- Add incremental sync from Google Calendar to K-CUBE.
- Add admin audit report and failure alerts.

Source References

- Google Identity Services and OAuth for web apps: <https://developers.google.com/identity/oauth2/web/guides/how-user-authz-works>
- Google OAuth 2.0 web server flow: <https://developers.google.com/identity/protocols/oauth2/web-server>
- Google Calendar events API: <https://developers.google.com/workspace/calendar/api/v3/reference/events/insert>
- Google Calendar sync guidance: <https://developers.google.com/workspace/calendar/api/guides/sync>
- Google Workspace domain-wide delegation: <https://support.google.com/a/answer/162106>